DOKUMENTASI PERANGKAT LUNAK

GUI Panel Kontrol Robot Feeder Akuatik

Pengembangan dengan Metode SDLC — Prototyping Model

Judul Skripsi	Pengembangan Robot Akuatik untuk Feeder Otomatis di Tambak
Sub-sistem	Transmitter — GUI Kendali (Raspberry Pi + NRF24L01)
Metode SDLC	Prototyping Model
Framework GUI	Python 3 + CustomTkinter
Target Tampilan	Layar sentuh 7" (800 × 480 px) — adaptif mengikuti layar perangkat
Protokol Kendali	FW / CW / CCW / STOP + ON & FIN (Dokumen Spesifikasi Teknis)
Tanggal	16 August 2026

1. Pendahuluan

1.1 Latar Belakang

Robot akuatik feeder otomatis telah berhasil beroperasi di air: mampu bergerak lurus dengan *heading-hold* berbasis MPU6050 dan berbelok melalui *differential thrust* dua motor brushless. Kendali sebelumnya dilakukan lewat antarmuka baris perintah (CLI) pada berkas **receiverArduino.py** yang mengharuskan operator mengetik dimensi tambak secara manual dan tidak menampilkan status sistem. Untuk meningkatkan kegunaan (*usability*) dan menyediakan pemantauan, dikembangkan antarmuka grafis (GUI) yang berjalan di Raspberry Pi.

1.2 Tujuan

- Menyediakan input dimensi tambak (panjang & lebar dalam meter) yang mudah.
- Menerima **titik awal robot**, **lebar lintasan**, dan **kecepatan**, lalu menerjemahkan tambak persegi menjadi **path/jalur** berupa daftar langkah "maju x m / belok y°" (pola zigzag) yang diikuti robot.
- Menerapkan **protokol Sinyal Kendali** sesuai Dokumen Spesifikasi Teknis: perintah **FW / CW / CCW / STOP** dengan pola Request-Response beserta sinyal balik **ON** dan **FIN**.
- Menyediakan **kendali manual** untuk memasukkan sendiri perintah maju (**dalam satuan meter**, dipecah otomatis menjadi paket FW) serta CW dan CCW di samping STOP yang sudah ada.
- Menyesuaikan tampilan dengan **ukuran layar perangkat** — dirancang untuk **layar sentuh 7 inci** bergaya mesin ATM: minimalis dan bertombol besar, namun seluruh fitur dipertahankan.
- Menyediakan **papan angka pada layar** agar dapat dioperasikan tanpa keyboard fisik.
- Menampilkan **preview jalur** pada kanvas (peta tambak + arah tempuh) dan daftar perintah yang akan dikirim sebelum misi dijalankan.
- Menampilkan status koneksi modul NRF24 dan keterjangkauan robot secara nyata.
- Memberi kontrol manual untuk mematikan komponen yang relevan.
- Menampilkan telemetri (estimasi) kecepatan & RPM baling-baling secara realtime.
- Menampilkan data sensor MPU (roll, pitch, yaw/heading) secara nyata — **dikirim robot lewat radio, tanpa port serial** — berjalan di latar belakang dengan saklar nyala/mati pencatatannya.
- Mencatat seluruh aktivitas, **menyimpannya otomatis ke berkas TXT sejak aplikasi dibuka**, membuang berkas yang lebih tua dari 3 hari, serta mengekspornya sebagai CSV/TXT.
- Menuntun operator lewat **alur kerja** (BERANDA → MISI → DASHBOARD) alih-alih menyodorkan menu berisi enam halaman.
- Memberi operator **pilihan mode kerja REAL atau SIMULASI** yang dapat dipindah kapan saja, dengan SIMULASI sebagai keadaan awal yang aman.

1.3 Ruang Lingkup & Batasan

GUI berjalan di sisi **Transmitter** (Raspberry Pi) sebagai pengganti CLI. Pada iterasi ini firmware robot (ESP32, **Finalnew.ino**) turut **ditulis ulang** agar perilakunya mengikuti Dokumen Spesifikasi Teknis: robot berubah dari penjalan misi mandiri menjadi **eksekutor perintah** yang menjalankan satu gerakan dalam satu waktu dan melapor lewat ON/FIN (lihat 4.2). Konsekuensi batasan teknis yang teridentifikasi pada tahap analisis:

- **Komunikasi dua arah half-duplex.** Radio NRF dipakai bergantian ("gantian" seperti radio HT): Pi mengirim perintah/PING, robot membalas ON/FIN dan telemetri MPU di sela-selanya (alamat balik 00002). Bukan full-duplex, jadi kirim/terima tidak bersamaan.

- **Tidak ada sensor RPM/encoder** pada robot. Maka nilai RPM & kecepatan yang ditampilkan adalah **ESTIMASI** yang dihitung dari nilai PWM throttle, bukan hasil pengukuran.
- **Jarak FW diturunkan dari waktu tempuh** (jarak ÷ kecepatan jelajah), bukan dari odometri. Ketepatan jarak karena itu bergantung pada kalibrasi ROBOT_SPEED_MPS dan dipengaruhi arus air.
- **ESC bersifat unidirectional** (hanya maju), sehingga CW/CCW dilakukan dengan *differential thrust*: robot berputar melingkar lebar, bukan di tempat. Sudut akhirnya tetap akurat karena putaran ditutup memakai umpan balik yaw MPU, tetapi robot ikut berpindah posisi selama berputar.
- **Kontrol komponen individual** (mis. mematikan satu motor atau MPU) belum memiliki perintah pada tabel protokol; ditandai sebagai indikator *monitor*.

Catatan: Kejujuran teknis ini justru menjadi bahan analisis yang baik: keterbatasan menjadi 'kebutuhan iterasi berikutnya' — sesuai semangat Prototyping.

2. Metodologi: SDLC Prototyping Model

Dipilih **Prototyping Model** karena kebutuhan GUI belum sepenuhnya pasti di awal dan paling baik ditemukan melalui purwarupa yang dievaluasi bersama dosen pembimbing. Cocok untuk sistem gabungan perangkat keras + lunak yang berevolusi.

2.1 Alasan Pemilihan (dibanding metode lain)

Metode	Kesesuaian untuk proyek ini
Waterfall	Rapi didokumentasikan, tetapi kaku bila kebutuhan berubah.
V-Model	Kuat di pengujian, tetapi mengasumsikan kebutuhan sudah pasti.
Incremental	Baik, mirip Prototyping namun tiap increment harus 'final'.
Prototyping (dipilih)	**Kebutuhan GUI ditemukan lewat purwarupa & umpan balik; iteratif; cocok HW+SW.**

2.2 Tahapan Prototyping

Fase	Aktivitas pada proyek ini
1. Pengumpulan Kebutuhan	Wawancara kebutuhan operator: input P×L, status NRF, kontrol komponen, telemetri realtime, ekspor log.
2. Perancangan Cepat (Quick Design)	Rancang tata letak GUI & pemisahan arsitektur core/GUI; definisikan protokol paket (misi/PING/STOP).
3. Bangun Prototipe	Implementasi feeder_core.py (RobotCommand, RadioLink, CommandRunner, ActivityLogger) dan feeder_gui.py (CustomTkinter).
4. Evaluasi oleh Pengguna	Demo ke pembimbing memakai MODE SIMULASI (tanpa hardware), lalu uji nyata di Raspberry Pi + robot.

Fase	Aktivitas pada proyek ini
5. Perbaikan (Refinement)	Revisi berdasar umpan balik: iterasi-1 (label 'estimasi', LED status, penanganan input); iterasi-2 (perencanaan jalur, titik awal & kecepatan, keypad layar, preview jalur, perbaikan pola belok); iterasi-3 (tampilan mengikuti layar perangkat bergaya ATM, perilaku robot mengikuti Dokumen Spesifikasi Teknis, kendali manual FW/CW/CCW). Ulangi fase 3–5.
6. Rekayasa Produk & Pemeliharaan	Stabilkan, dokumentasikan, siapkan iterasi lanjutan (odometri & sensor RPM).

2.3 Siklus Iterasi (Prototyping Loop)

```
Kebutuhan -> Quick Design -> Bangun Prototipe
^ |
| v
Perbaikan <---- Evaluasi Pengguna <---+
(ulangi hingga pengguna puas -> Produk Final)
```

3. Analisis Kebutuhan

3.1 Kebutuhan Fungsional

Kode	Kebutuhan	Status
RF-1	Input panjang & lebar tambak (meter) dan kirim misi AUTO	Terpenuhi
RF-2	Menampilkan status modul NRF & keterjangkauan robot (ACK)	Terpenuhi (real)
RF-3	Mematikan komponen yang dapat dikontrol (NRF TX, PING, Motor)	Terpenuhi
RF-4	Menampilkan RPM & kecepatan baling-baling realtime	Terpenuhi (estimasi)
RF-5	Mencatat & mengekspor log aktivitas (CSV/TXT)	Terpenuhi
RF-6	Perintah STOP membatalkan misi berjalan	Terpenuhi (real)
RF-7	Menampilkan data MPU6050 (roll/pitch/yaw) tanpa port serial — mengalir sendiri lewat telemetri radio	Terpenuhi (real)
RF-8	Input titik awal robot (sudut tambak) & kecepatan & lebar lintasan	Terpenuhi
RF-9	Menerjemahkan path jadi daftar langkah "maju x / belok y°" & preview kanvas	Terpenuhi
RF-10	Keypad numerik pada layar (operasi tanpa keyboard fisik)	Terpenuhi
RF-11	Mengirim parameter misi ke robot (kecepatan, lebar lintasan, arah belok)	Terpenuhi (real)
RF-12	Menerima telemetri MPU (roll/pitch/yaw) balik dari robot via radio (2 arah)	Terpenuhi (real)
RF-13	Protokol Request-Response "[Moves]:[Dist]" dengan sinyal balik ON & FIN sesuai Dokumen Spesifikasi Teknis	Terpenuhi (real)
RF-14	Mengirim ulang perintah bila "ON [Moves]" tak diterima dalam 1 detik	Terpenuhi (real)
RF-15	Input manual perintah FW, CW, dan CCW (STOP sudah ada)	Terpenuhi (real)

Kode	Kebutuhan	Status
RF-16	Menolak [Dist] di luar rentang tabel perintah sebelum paket dikirim	Terpenuhi
RF-17	Tampilan mengikuti ukuran layar perangkat (layar sentuh 7 inci, gaya ATM)	Terpenuhi
RF-18	Perintah maju manual diisi dalam **meter** lalu dipecah otomatis menjadi paket FW 10–200 cm	Terpenuhi
RF-19	Saklar nyala/mati **pencatatan sudut MPU** ke log	Terpenuhi
RF-20	Saklar **auto-simpan log ke berkas TXT** selama operasi berjalan	Terpenuhi
RF-21	Pemilihan **mode kerja REAL/SIMULASI** saat aplikasi berjalan (default SIMULASI)	Terpenuhi
RF-22	Layar pembuka hanya berisi **MISI** & **SETTING**; DASHBOARD terbuka otomatis sesudah misi dikirim	Terpenuhi
RF-23	DASHBOARD menampilkan **koordinat sudut MPU** robot berjalan beserta tombol berhenti berukuran besar	Terpenuhi
RF-24	Auto-simpan log **menyala sejak aplikasi dibuka** (tanpa perlu dinyalakan operator)	Terpenuhi
RF-25	Berkas log lebih tua dari **3 hari dihapus otomatis** agar kartu SD Raspberry Pi tidak penuh	Terpenuhi

3.2 Kebutuhan Non-Fungsional

- **Keterandalan:** heartbeat PING tiap 0,5 s menjaga failsafe robot (stop < 3 s bila sinyal hilang).
- **Portabilitas:** berjalan di Raspberry Pi (NRF real) & PC; mode SIMULASI tidak menuntut pustaka perangkat keras apa pun.
- **Keselamatan operasi:** aplikasi selalu terbuka di mode SIMULASI sehingga robot tidak pernah bergerak tanpa keputusan sadar operator; perpindahan mode ditolak selagi ada perintah berjalan.
- **Ketahanan catatan:** auto-simpan menulis tiap baris log begitu terjadi, sehingga data uji lapangan tidak hilang bila aplikasi ditutup paksa atau daya mati.
- **Hemat penyimpanan:** log MPU menulis sekitar satu baris per detik, maka berkas berumur lebih dari 3 hari dibuang otomatis — kartu SD Raspberry Pi tidak menggelembung walau alat dipakai berhari-hari.
- **Usability:** indikator warna (hijau/oranye/merah), label satuan jelas, serta **operasi penuh lewat sentuhan** (keypad di layar + preview jalur) tanpa keyboard/mouse.
- **Kejujuran data:** nilai estimasi diberi label eksplisit agar tidak menyesatkan.

4. Perancangan (Quick Design)

4.1 Arsitektur Perangkat Lunak

Menerapkan *Separation of Concerns*: lapisan presentasi (GUI) dipisah dari logika (core), sehingga mudah diuji dan dikembangkan.

Modul	Tanggung jawab
feeder_gui.py	Presentasi: halaman bergaya ATM, papan angka layar, kanvas preview jalur, tombol, gauge, LED, tampilan log.

Modul	Tanggung jawab
feeder_core.py -> RobotCommand	Satu paket Sinyal Kendali "[Moves]:[Dist]" + validasi rentang sesuai tabel perintah dokumen spesifikasi.
feeder_core.py -> plan_coverage / plan_command_queue	Terjemahkan tambak + titik awal + lebar lintasan -> daftar segmen "maju/belok" -> ANTREAN perintah FW/CW/CCW.
feeder_core.py -> RadioLink	Abstraksi NRF24 (pyrf24) dan MockRadio (simulasi) yang dapat DIPINDAH saat berjalan lewat set_mode(); merutekan sinyal balik ON/FIN dan telemetri MPU.
feeder_core.py -> CommandRunner	Pelaksana pola Request-Response: kirim perintah, tunggu ON (kirim ulang bila lewat 1 detik), tunggu FIN, lanjut perintah berikutnya.
feeder_core.py -> ActivityLogger	Pencatatan aktivitas berstempel waktu, ekspor CSV/TXT, dan auto-simpan tiap baris ke berkas TXT.
feeder_core.py -> MPUMonitor	Mengambil sudut MPU dari telemetri NRF di latar belakang, menyaring paket kembar, dan mencatatnya ke log bila saklar Log MPU menyala.

4.2 Protokol Sinyal Kendali (mengikuti Dokumen Spesifikasi Teknis)

Protokol mengikuti bab **Skema Komunikasi** pada *Dokumen Spesifikasi Teknis — Robot Akuatik untuk Tambak Ikan*, yang mengadopsi skema **DJI Tello SDK 2.0**. Sinyal Kendali **selalu diinisiasi Pengendali Utama**; Body Robot memberi respon. Setiap perintah dihitung sebagai satu paket komunikasi dengan pola **Request-Response**.

Paket	Arah	Makna
[Moves]:[Dist]	Pi -> Robot	Perintah gerak. Contoh "FW:100" = maju lurus 100 cm; "CCW:90" = berputar berlawanan jarum jam 90°.
ON [Moves]	Robot -> Pi	Dikirim berkali-kali dengan jeda 1 detik SELAMA gerakan berlangsung, agar Pengendali Utama tahu perintah sedang dijalankan.
FIN [Moves]:[Dist]	Robot -> Pi	Gerakan selesai dieksekusi dengan baik. Menjadi izin bagi Pengendali Utama untuk mengirim perintah berikutnya.
PING	Pi -> Robot	Heartbeat (jaga failsafe robot) tiap 0,5 s + pemicu giliran bicara robot
(ACK otomatis)	Robot -> Pi	Auto-ACK NRF; True = robot hidup & terjangkau
R=<roll>;P=<pitch>;Y=<yaw>;S=<0|1>	Robot -> Pi	Telemetri MPU (tambahan di luar spesifikasi wajib) — alamat balik 00002, NOACK. S=1 sudut sah, S=0 MPU tidak menjawab I2C

Daftar perintah beserta rentang [Dist] yang sah (tabel perintah pada dokumen spesifikasi). GUI menolak nilai di luar rentang ini sebelum paket dikirim, dan firmware memvalidasinya sekali lagi saat paket diterima:

No	Moves	Dist Range	Deskripsi
1	FW	10–200	Maju lurus ke depan (satuan sentimeter)

No	Moves	Dist Range	Deskripsi
2	CW	1–180	Berputar searah jarum jam / ke kanan (satuan derajat)
3	CCW	1–180	Berputar berlawanan arah jarum jam / ke kiri (satuan derajat)
4	STOP	0	Menghentikan motor. Digunakan untuk berhenti secara darurat.

Penanganan kegagalan. Bila Pengendali Utama tidak menerima **ON [Moves]** dalam **1 detik**, perintah dianggap gagal dijalankan dan **dikirim ulang** (hingga 5 percobaan). Agar paket kiriman ulang tidak menjadi gerakan ganda, robot mengabaikan perintah gerak baru selama masih ada gerakan berjalan — kecuali STOP, yang selalu dilayani seketika.

```
Pi -> FW:200 (kirim perintah)
Pi <- ON FW (< 1 detik, kalau tidak -> kirim ulang)
Pi <- ON FW (berulang tiap 1 detik selama bergerak)
Pi <- FIN FW:200 (gerakan selesai -> lanjut perintah ke-2)
```

Perubahan peran robot. Pada iterasi sebelumnya robot menerima satu paket misi (P/L/MODE) lalu menjalankan seluruh pola zigzag sendiri di dalam firmware. Kini seluruh pola disusun dan dikemudikan **Pengendali Utama**, sedangkan robot menjadi **eksekutor satu perintah** yang melapor lewat ON/FIN. Konsekuensinya kemajuan misi yang ditampilkan GUI adalah **konfirmasi nyata dari robot**, bukan lagi perkiraan berbasis penghitung waktu.

Komunikasi 2 arah half-duplex ("gantian"). NRF24 tidak bisa kirim & terima bersamaan, jadi kedua sisi bergantian seperti radio HT: Pi mengirim paket (mode TX) lalu mendengar sejenak (mode RX); robot yang menerima paket sebentar *stopListening*, mengirim ON/FIN/telemetri tanpa ACK, lalu *startListening* lagi. Window TX yang sangat pendek menjaga perintah tidak terlewat.

4.3 Dasar Estimasi Telemetri Baling-baling

Robot tidak memiliki sensor RPM, sehingga gauge baling-baling tetap berupa **estimasi**. Bedanya dengan iterasi lalu: estimasi kini diturunkan dari **perintah yang sedang benar-benar dijalankan** (diketahui dari sinyal ON/FIN), bukan dari cermin penghitung waktu firmware.

Perintah berjalan	PWM kiri	PWM kanan	Alasan
FW	CRUISE	CRUISE	Kedua motor maju
CW (kanan)	CRUISE	STOP	Differential thrust ke kanan
CCW (kiri)	STOP	CRUISE	Differential thrust ke kiri
tidak ada / STOP	STOP	STOP	Motor diam

```
throttle = (PWM - 1000) / (2000 - 1000)
RPM_est = throttle * MOTOR_KV * BATTERY_V (default KV=2200, V=11.1)
speed_est= ROBOT_SPEED_MPS * throttle / throttle(CRUISE_PWM)
```

> **Catatan:** Konstanta MOTOR_KV, BATTERY_V, CRUISE_PWM, dan ROBOT_SPEED_MPS wajib disamakan dengan firmware & dikalibrasi pada pengujian air agar estimasi mendekati kenyataan.

4.4 Integrasi Data MPU6050 (lewat Telemetri Radio, tanpa Port Serial)

Data orientasi diambil **langsung dari sensor MPU6050** (bukan estimasi). Pada iterasi ini jalur **serial USB dihapus**: menuntut operator memilih port (*/dev/ttyUSB0*, *COM11*, …) hanya masuk akal saat board tertambat kabel di meja, padahal robot bekerja mengapung. Sudut kini **dikirim robot sendiri**

menumpang telemetri balik NRF setiap kali Pengendali Utama mengirim PING, sehingga mengalir di **latar belakang** selama heartbeat hidup — tanpa satu pun pengaturan di layar.

```
Robot --(NRF, alamat 00002)--> RadioLink.pump() --> get_telemetry()
|
MPUMonitor.poll() --> tampilan + log
```

Format yang di-parse: **R=<roll>;P=<pitch>;Y=<yaw>;S=<0|1>**. Bidang **S** adalah status kesehatan MPU yang dilaporkan firmware: bila robot tidak dapat membaca sensornya lewat I2C, sudut yang terkirim hanyalah nilai beku (khasnya Roll=-90,0 / Pitch=90,0 / Yaw=-0,0) sehingga GUI menampilkannya **merah disertai peringatan**, bukan sebagai data sungguhan. Bidang S berada di ekor paket agar firmware lama yang belum mengirimnya tetap dikenali. Kelas **MPUMonitor** menyimpan cuplikan terakhir beserta sumber & umurnya, **menyaring paket kembar** (poll dipanggil jauh lebih sering daripada laju telemetri, jadi paket yang sama akan terlihat berkali-kali), lalu mencatat satu baris per detik ke **ActivityLogger** bila saklar **Log MPU** dinyalakan operator. Karena pencatatan berada di lapisan core — bukan di halaman SENSOR — log tetap berjalan walau operator sedang membuka halaman lain, dan ikut tersimpan ke berkas TXT bila auto-simpan menyala.

> **Catatan:** Konsekuensinya **pyserial tidak lagi menjadi dependensi**, dan halaman SENSOR kehilangan kolom port beserta tombol Hubungkan/Putus — satu bidang input yang hilang, satu sumber kesalahan operator yang hilang pula.

4.5 Mode Kerja REAL / SIMULASI sebagai Pilihan Operator

Sebelumnya mode ditentukan **otomatis**: bila pyrf24 ada dipakai NRF nyata, bila tidak jatuh ke simulasi. Perilaku itu membuat operator tidak pernah memutuskan apa pun — padahal di Raspberry Pi hasilnya **selalu REAL**, sehingga latihan atau demo di darat berisiko benar-benar menggerakkan motor. Kini mode menjadi **pilihan sadar**:

- **Keadaan awal selalu SIMULASI** (robot virtual) di perangkat mana pun.
- Tombol **MODE** di bilah status atas — terlihat dari semua halaman — memindahkan mode dengan satu sentuhan; pemilih **MODE KERJA** di halaman SISTEM melakukan hal yang sama dan keduanya selalu sinkron.
- Flag **--real** tersedia bagi pemakaian rutin di lapangan agar tidak perlu menyentuh apa pun setelah aplikasi terbuka.
- **RadioLink.set_mode()** melepas radio lama, membersihkan sisa ON/FIN & telemetri, lalu memasang lawan bicara baru — sehingga data antar-mode tidak tercampur.
- Bila REAL diminta tetapi pyrf24/NRF tidak tersedia, sistem kembali ke SIMULASI dan **melaporkan sebabnya** lewat dialog dan log (tidak diam-diam).
- Perpindahan mode **ditolak selagi ada perintah berjalan**; operator diminta menekan STOP dahulu agar robot tidak ditinggalkan bergerak tanpa pengendali.

4.6 Pencatatan Aktivitas & Auto-Simpan TXT

Ekspor CSV/TXT hanya menyimpan **saat tombol ditekan**, sehingga catatan uji lapangan bisa lenyap bila aplikasi ditutup paksa atau Raspberry Pi kehilangan daya. Karena itu **ActivityLogger** dilengkapi **auto-simpan**: saklar pada halaman LOG membuat berkas **logs/log_feeder_<tanggal>_<jam>.txt**, menuliskan catatan yang sudah ada, lalu menulis **tiap baris baru begitu terjadi**. Berkas dibuka mode *append* dan ditutup setiap baris supaya isinya benar-benar sudah berada di kartu SD, bukan menunggu di penyangga memori — biaya yang tak berarti pada laju sekitar satu baris per detik.

Bila penulisan gagal di tengah operasi (kartu penuh, media hanya-baca), auto-simpan dimatikan sendiri dengan sebabnya disimpan; GUI menarik keadaan itu tiap tick lalu mengembalikan saklar ke posisi mati dan menampilkan alasannya — sehingga tidak ada layar yang mengaku 'aktif' padahal tidak ada yang tersimpan.

Menyala sejak start & membersihkan diri. Menuruti arahan pembimbing, pencatatan tidak lagi menunggu ditekan: begitu aplikasi dibuka, auto-simpan TXT dan log MPU langsung aktif, sehingga setiap uji lapangan pasti meninggalkan berkas. Konsekuensinya folder log tumbuh terus — log MPU saja menulis sekitar satu baris per detik — maka **purge_old_logs()** membuang berkas yang lebih tua dari **LOG_RETENTION_DAYS = 3 hari** saat aplikasi dibuka dan mengulangnya tiap 6 jam selagi aplikasi hidup. Berkas yang *sedang* ditulis selalu dikecualikan, sehingga sesi berjalan tidak mungkin ikut terhapus walau jam sistem Raspberry Pi melompat (kejadian lumrah pada papan tanpa RTC).

```
start aplikasi
-> purge_old_logs(): hapus logs/*.txt umur > 3 hari
-> set_autosave(logs/log_feeder_<tgl>_<jam>.txt)
-> MPUMonitor.set_logging(True)
-> ulangi pemeriksaan retensi tiap 6 jam
```

4.7 Perencanaan Jalur & Penyusunan Antrean Perintah

Fungsi **plan_coverage()** menerjemahkan tambak persegi menjadi jalur sapuan *boustrophedon* (zigzag). Dari masukan pengguna — panjang, lebar, **lebar lintasan**, **kecepatan**, dan **titik awal** (sudut tambak) — dihitung titik-titik belok (waypoint), lalu diturunkan menjadi daftar langkah eksplisit "maju x meter" dan "belok kanan/kiri y°".

Selanjutnya **plan_command_queue()** mengubah daftar langkah itu menjadi **antrean paket Sinyal Kendali**. Karena rentang FW dibatasi 10–200 cm, lintasan panjang otomatis **dipecah rata** menjadi beberapa perintah FW berurutan — pembagian dibuat rata agar tidak menyisakan potongan di bawah batas minimum (mis. 10,05 m menjadi 6×167 cm, bukan 5×200 cm + sisa 5 cm yang tak bisa diperintahkan). Belok kanan menjadi **CW**, belok kiri menjadi **CCW**.

Koordinat: sumbu-x menyusuri lebar, sumbu-y menyusuri panjang. Robot menjelajah lurus arah-y tiap lintasan, lalu U-turn menggeser sejauh **satu spasi jalur** arah-x.

Jalur pertama dan terakhir menempel di tepi tambak. Lebar dibagi menjadi **nBands = ■lebar / lebar lintasan■** pita selebar **spasi = lebar / nBands** (selalu ≤ lebar lintasan yang diminta), lalu jalur ditaruh di **setiap garis pembatas pita** — termasuk x = 0 dan x = lebar. Jumlah jalur karena itu **nBands + 1**. Pada tambak 2×2 m dengan lintasan 1 m dihasilkan 3 jalur di x = 0, 1, dan 2 m, sehingga lintasan benar-benar mencapai ujung kotak di kedua sisi. Kekeliruan yang mudah terjadi adalah menaruh jalur pertama di tepi lalu menggesernya sejauh lebar lintasan *tanpa menambah jumlah jalur*: sisi seberang tidak pernah dijadikan jalur dan tambak 2×2 m hanya tersapu separuh.

> **Catatan:** Konsekuensi yang disengaja: robot melintas tepat di sisi dinding tambak dan separuh sapuan jalur tepi jatuh di luar tambak. Bila yang diutamakan adalah jarak aman dari dinding, jalur perlu digeser masuk setengah spasi — dengan konsekuensi lintasan tidak lagi menyentuh ujung kotak.

```
Contoh: tambak 10 x 4 m, lebar lintasan 1 m, mulai Kiri-Bawah
nBands = 4, spasi = 4/4 = 1 m -> 5 lintasan di x = 0/1/2/3/4 m

Langkah jalur Antrean perintah kendali (37 paket)
1. Maju 10.0 m -> FW:200 FW:200 FW:200 FW:200 FW:200
2. Belok kanan 90 -> CW:90
3. Maju 1.0 m -> FW:100
4. Belok kanan 90 -> CW:90
```

```
5. Maju 10.0 m -> FW:200 x5
6. Belok kiri 90 -> CCW:90
... (dst hingga Lane 5 di tepi kanan)
```

Pola belok berselang-seling. U-turn yang **selalu belok kanan** secara geometri menggeser robot ke kanan lalu ke kiri bergantian, sehingga robot hanya bolak-balik di dua kolom dan **tidak menyapu seluruh lebar tambak**. Arah U-turn karena itu dibuat **berselang-seling** (kanan-kanan, lalu kiri-kiri, dst) sehingga geseran selalu ke sisi yang sama. Karena antrean perintah dihitung dari jalur yang sama dengan yang digambar di layar, **preview jalur = gerak nyata robot**.

> **Catatan:** Titik awal dapat dipilih lewat menu atau **disentuh langsung pada kanvas peta**; arah belok pertama dihitung dari titik awal agar robot menyapu ke arah dalam tambak.

4.8 Rancangan Antarmuka: Alur Kerja, Bukan Menu

Menindaklanjuti umpan balik pembimbing (dua putaran evaluasi), tata letak dirancang ulang agar **mengikuti ukuran layar perangkat** dan nyaman disentuh pada **layar sentuh 7 inci** (800 × 480 piksel) — dengan gaya **mesin ATM**: minimalis, target sentuh besar, satu layar untuk satu tugas, namun **seluruh fitur dipertahankan**.

- **Fullscreen adaptif.** Jendela mengambil resolusi layar perangkat (*winfo_screenwidth/height*) dan berjalan tanpa bilah judul. Ukuran huruf, tinggi tombol, dan jarak antar elemen dikalikan faktor skala $S = min(lebar/800, tinggi/480)$ yang dijepit pada rentang 0,85–2,0 sehingga tampilan tetap proporsional di layar 7 inci maupun monitor besar.
- **Dari menu enam tombol menjadi alur.** Iterasi sebelumnya memakai bilah navigasi berisi enam halaman. Umpan balik pembimbing: operator tidak perlu memilih "mau ke halaman mana" — ia hanya punya dua urusan, yaitu **menjalankan misi** atau **mengatur alat**. Bilah navigasi karena itu DIHAPUS: beranda menyisakan dua tombol besar (MISI & SETTING), dan **DASHBOARD terbuka sendiri** begitu misi dikirim karena itulah yang ingin dilihat operator pada saat itu.
- **Tidak ada fitur yang hilang.** Halaman MANUAL, LOG, dan SISTEM berkumpul sebagai tiga sub-halaman SETTING yang dipilih lewat satu pemilih segmen; halaman SENSOR melebur ke DASHBOARD karena isinya memang untuk dipantau saat robot bergerak.
- **Satu jalan pulang.** Tombol ‘<’ di kiri bilah status menggantikan seluruh bilah navigasi — disembunyikan di BERANDA karena di sana tidak ada tempat untuk kembali.
- **Tombol STOP permanen.** Berhenti darurat diletakkan di bilah status atas sehingga **selalu terlihat dan terjangkau dari halaman mana pun**.
- **Papan angka sebagai lapisan penuh.** Menyentuh sebuah kolom membuka papan angka satu layar penuh dengan tombol besar dan tombol OK / HAPUS / BATAL, sehingga halaman tetap lega tanpa kehilangan kemampuan input tanpa keyboard.

```
BERANDA MISI DASHBOARD (berjalan)
+-----------+ -> +---------------+ -> +-------------------+
| MISI | | ukuran tambak | | status + progres |
+-----------+ | preview jalur | | sudut MPU (real) |
| SETTING | | [KIRIM MISI] | | [ HENTIKAN/STOP ] |
+-----------+ +---------------+ +-------------------+

+-------------------------------------------------+
| [<] judul [MODE*][NRF][ROBOT] [ STOP ] | bilah status
+-------------------------------------------------+
| ISI HALAMAN |
+-------------------------------------------------+
```

```
* chip MODE dapat disentuh: REAL <-> SIMULASI
```

Halaman	Isi
BERANDA	Dua tombol besar: **MISI** dan **SETTING**. Selagi ada misi berjalan muncul jalan pintas **LIHAT DASHBOARD**.
MISI	Panjang, lebar, lebar lintasan, kecepatan, titik awal; preview jalur pada kanvas; daftar antrean perintah; tombol KIRIM MISI (→ DASHBOARD).
DASHBOARD	Terbuka otomatis sesudah misi/perintah dikirim: status & perintah berjalan, kemajuan antrean + sisa waktu, **koordinat sudut MPU (Roll/Pitch/Yaw)** yang dikirim robot, gauge baling-baling (estimasi), tombol HENTIKAN (STOP) berukuran besar, dan tombol kembali ke BERANDA.
SETTING › SISTEM	Saklar komponen (NRF, Heartbeat, Motor), indikator monitor, pemilih **MODE KERJA** (SIMULASI/REAL), status log & auto-simpan, informasi layar, dan tombol keluar.
SETTING › MANUAL	Maju dalam **meter** + putar kiri/kanan dalam derajat: nilai besar, tombol –/+, papan angka, penunjuk jumlah paket FW, dan tombol KIRIM per perintah (→ DASHBOARD).
SETTING › LOG	Catatan aktivitas realtime, saklar **Auto-simpan TXT** & **Log MPU** (keduanya menyala sejak aplikasi dibuka), keterangan retensi 3 hari, serta Export CSV / Export TXT / Clear.

5. Implementasi Prototipe

5.1 Fitur GUI

- **Halaman BERANDA:** layar pembuka bergaya ATM berisi dua tombol besar — MISI dan SETTING — ditambah jalan pintas LIHAT DASHBOARD yang hanya muncul selagi ada perintah berjalan.
- **Halaman DASHBOARD:** status robot berukuran besar (SIAP / MENGIRIM PERINTAH / ROBOT BERGERAK / KIRIM ULANG / SELESAI / DIBATALKAN / GAGAL), paket perintah yang sedang dijalankan, kemajuan antrean, perkiraan sisa waktu, koordinat sudut MPU (Roll/Pitch/Yaw), gauge baling-baling, dan tombol HENTIKAN berukuran besar. Terbuka **otomatis** sesudah KIRIM MISI atau KIRIM perintah manual.
- **Halaman MISI:** input Panjang, Lebar, Lebar lintasan, Kecepatan (m/s), dan Titik awal; kanvas preview jalur zigzag berpanah + titik START (sentuh peta = pilih titik awal); daftar **antrean perintah kendali** yang akan dikirim beserta ringkasan (jumlah lintasan, jarak, jumlah perintah, estimasi waktu).
- **SETTING › MANUAL:** tiga kartu besar berisi nilai, tombol –/+, papan angka, dan tombol KIRIM. Kartu **MAJU diisi dalam METER** (0,1–50 m, menerima pecahan seperti 2,5 m) dan di bawah angkanya tertulis menjadi berapa paket FW jarak itu dipecah — operator melihat persis apa yang akan dikirim. Kartu PUTAR KIRI/KANAN tetap derajat (1–180) sesuai protokol. Rentang sah ditampilkan dan divalidasi; nilai di luar rentang ditolak sebelum dikirim.
- **Papan angka layar penuh:** tombol 0–9, titik, backspace, HAPUS, BATAL, OK — muncul saat kolom disentuh; memungkinkan operasi **tanpa keyboard fisik** pada layar sentuh.
- **Status Koneksi:** chip MODE (REAL/SIMULASI) yang **dapat disentuh untuk berpindah mode**, MODUL NRF, dan ROBOT pada bilah atas — terlihat di semua halaman.

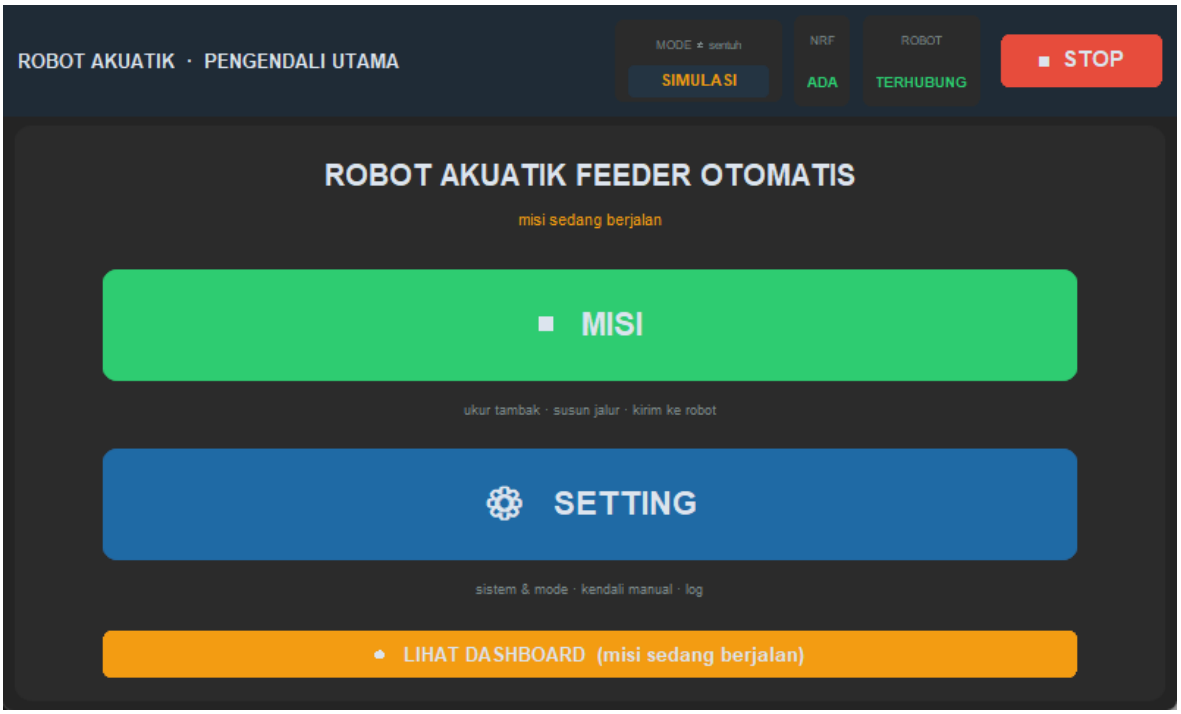
- **Tombol STOP permanen** pada bilah atas: mengirim **STOP:0** dan membatalkan sisa antrean dari halaman mana pun.
- **SETTING › LOG:** catatan realtime seluruh siklus protokol (KIRIM → GERAK → SELESAI), saklar **Auto-simpan TXT** dan **Log MPU** yang keduanya sudah menyala sejak aplikasi dibuka, nama berkas tujuan, keterangan retensi 3 hari, serta tombol Export CSV / Export TXT / Clear.
- **SETTING › SISTEM:** saklar NRF TX, Heartbeat PING, dan Motor (efek nyata lewat STOP); indikator monitor Motor Kiri/Kanan & MPU; pemilih **MODE KERJA** REAL/SIMULASI dan saklar robot virtual; ringkasan status log MPU & auto-simpan; informasi resolusi layar & faktor skala.

5.2 Mode Simulasi

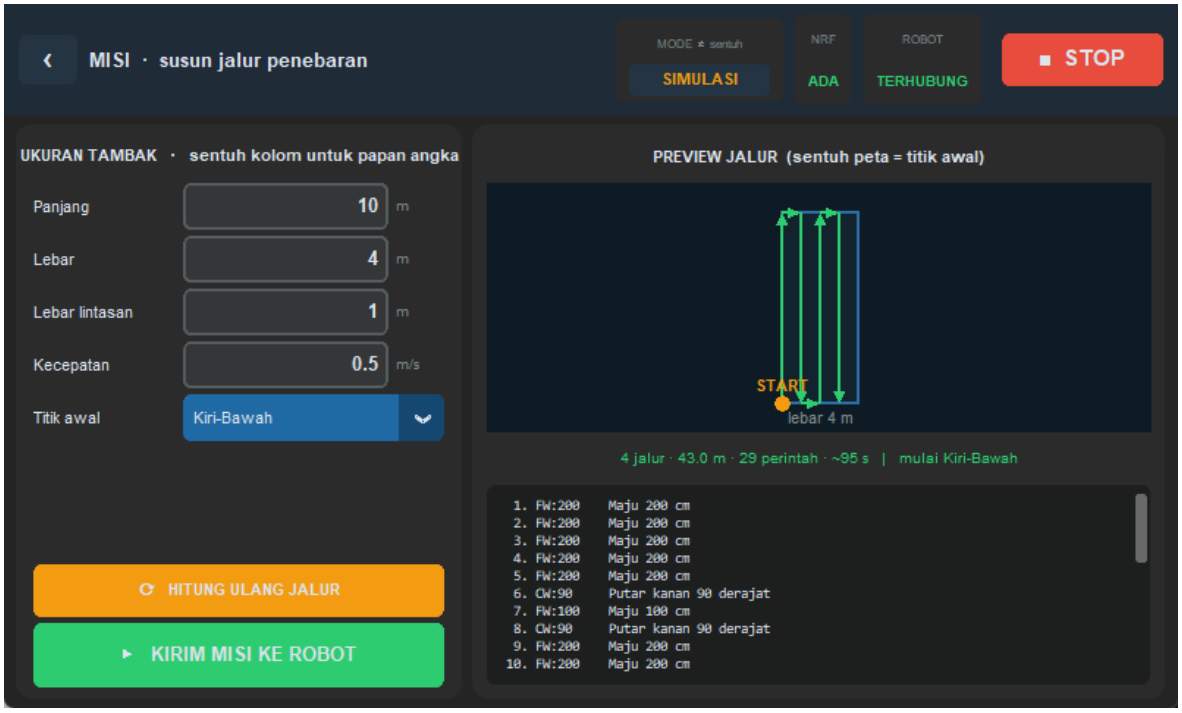
Mode SIMULASI kini menjadi **keadaan awal aplikasi**, bukan sekadar jaring pengaman ketika pyrf24 tidak ada: RadioLink dimulai memakai **MockRadio** dan baru menyentuh perangkat keras setelah operator memilih REAL. MockRadio **meniru perilaku Body Robot secara penuh**: menerima [Moves]:[Dist], membalas "ON [Moves]" tiap 1 detik, lalu "FIN [Moves]:[Dist]" saat gerakan selesai, dan berhenti seketika saat menerima STOP. Dengan begitu **seluruh alur Request-Response** dapat diuji dan didemokan tanpa perangkat keras — sangat membantu pada fase evaluasi Prototyping.

5.3 Tampilan Antarmuka

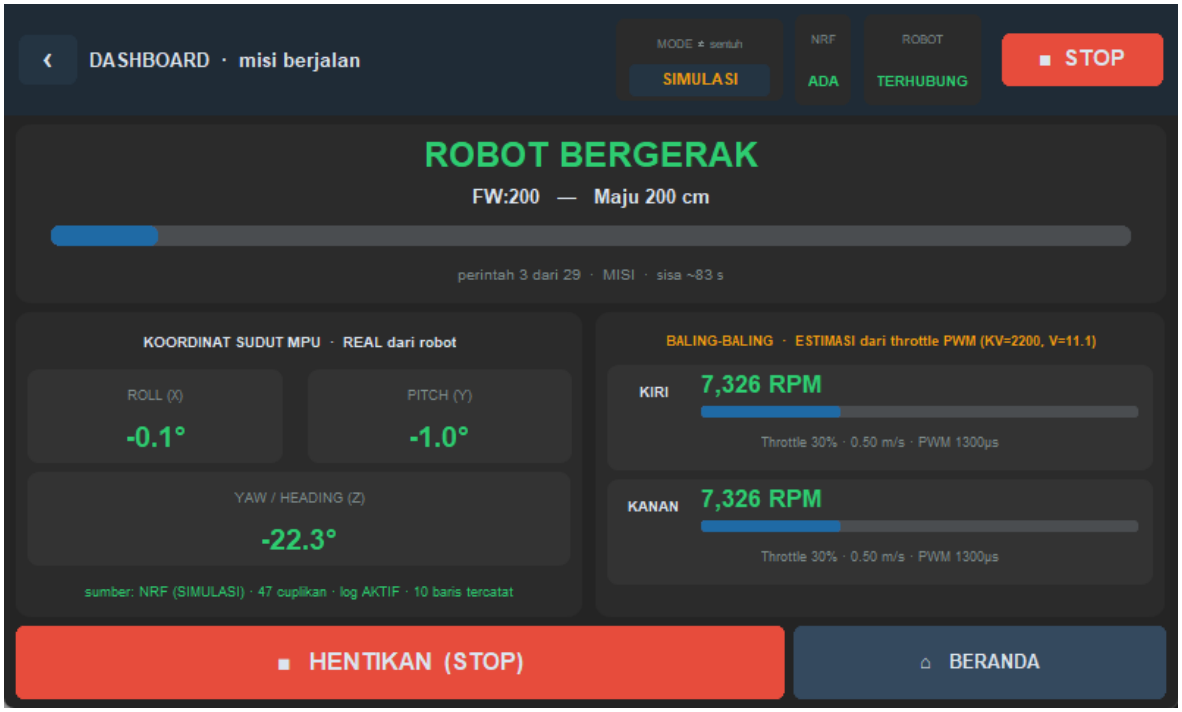
Seluruh tangkapan layar berikut diambil pada ukuran **800 × 480 piksel**, yaitu resolusi layar sentuh 7 inci yang dipakai Pengendali Utama.



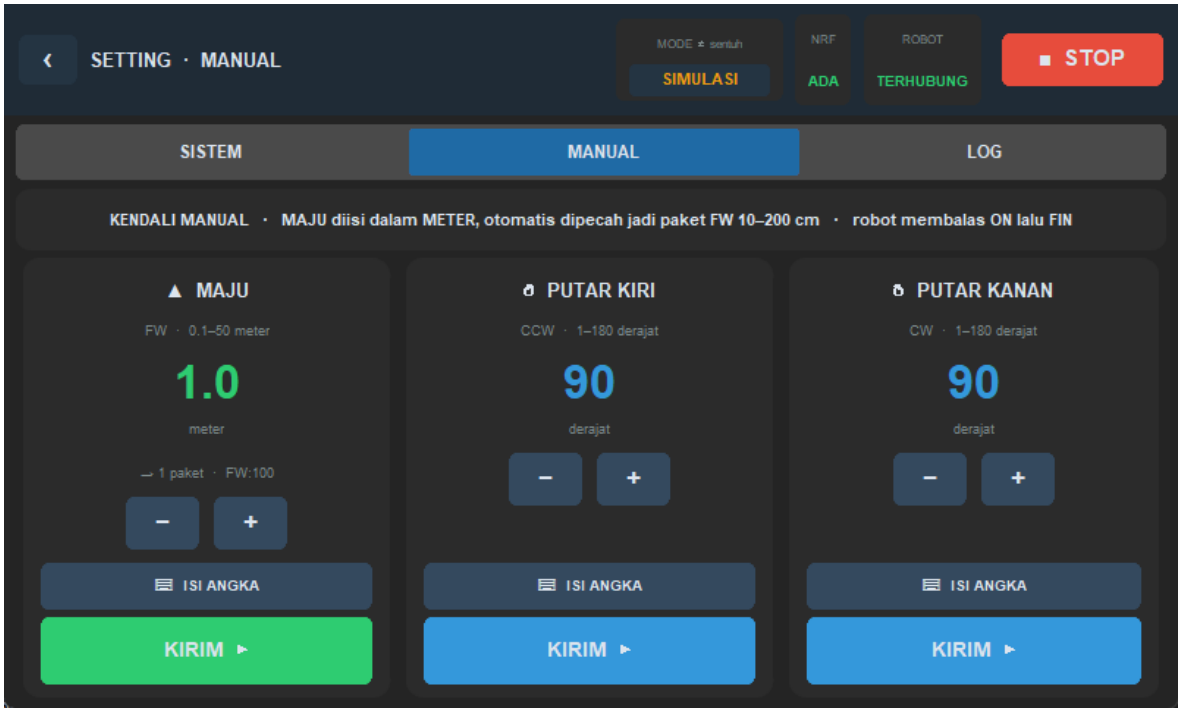
Gambar 1. Halaman BERANDA: layar pembuka hanya menawarkan MISI dan SETTING; jalan pintas LIHAT DASHBOARD muncul karena ada misi yang sedang berjalan.



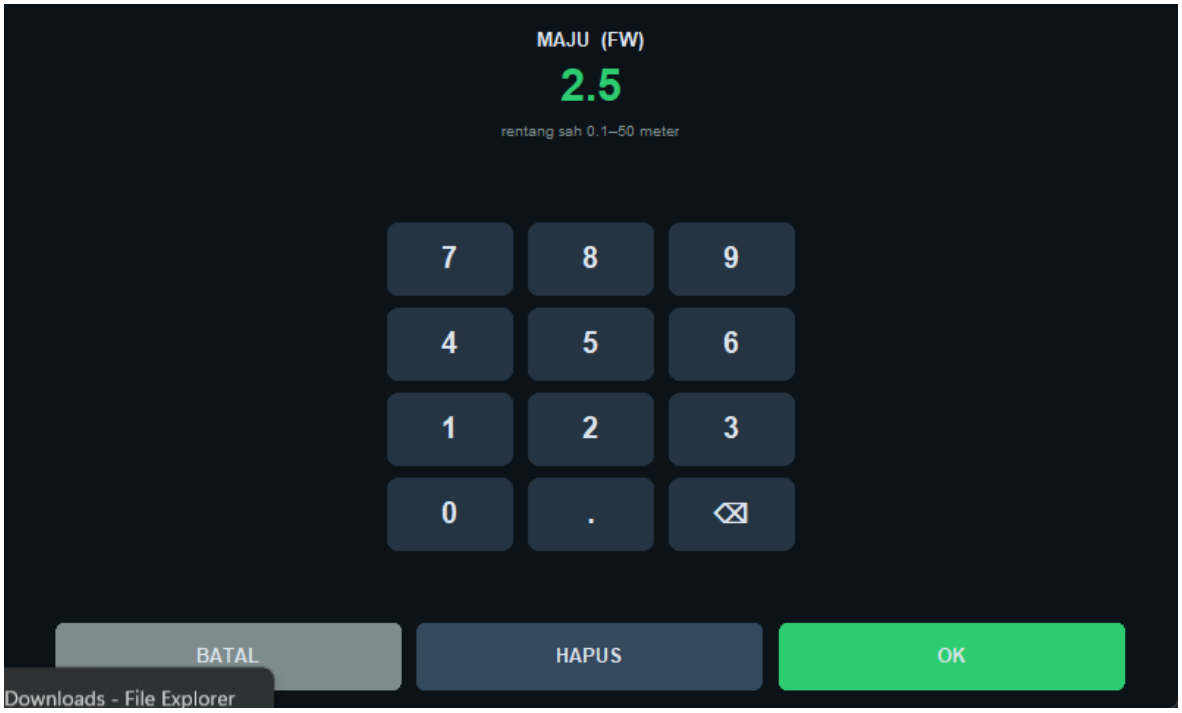
Gambar 2. Halaman MISI: input dimensi tambak di kiri; preview jalur zigzag dan daftar antrean perintah kendali (FW/CW/CCW) yang akan dikirim di kanan.



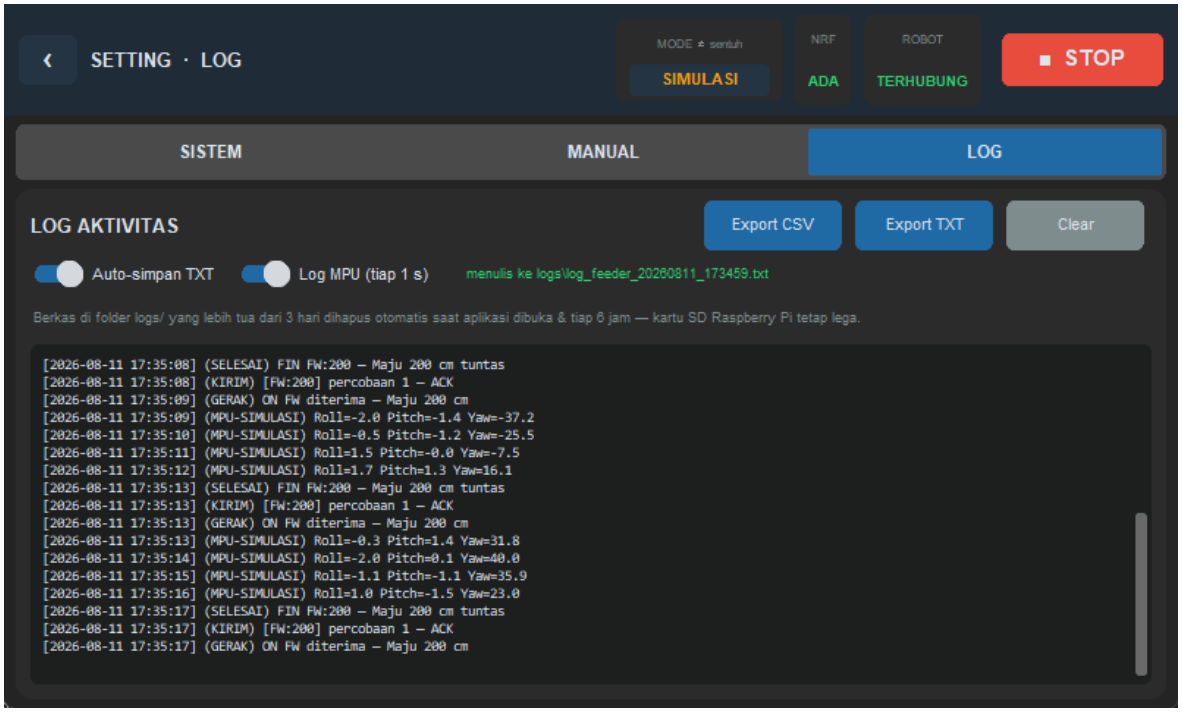
Gambar 3. Halaman DASHBOARD — terbuka sendiri sesudah KIRIM MISI: status dan perintah yang sedang dijalankan, kemajuan antrean, koordinat sudut MPU dari robot, gauge baling-baling (estimasi), serta tombol HENTIKAN berukuran besar.



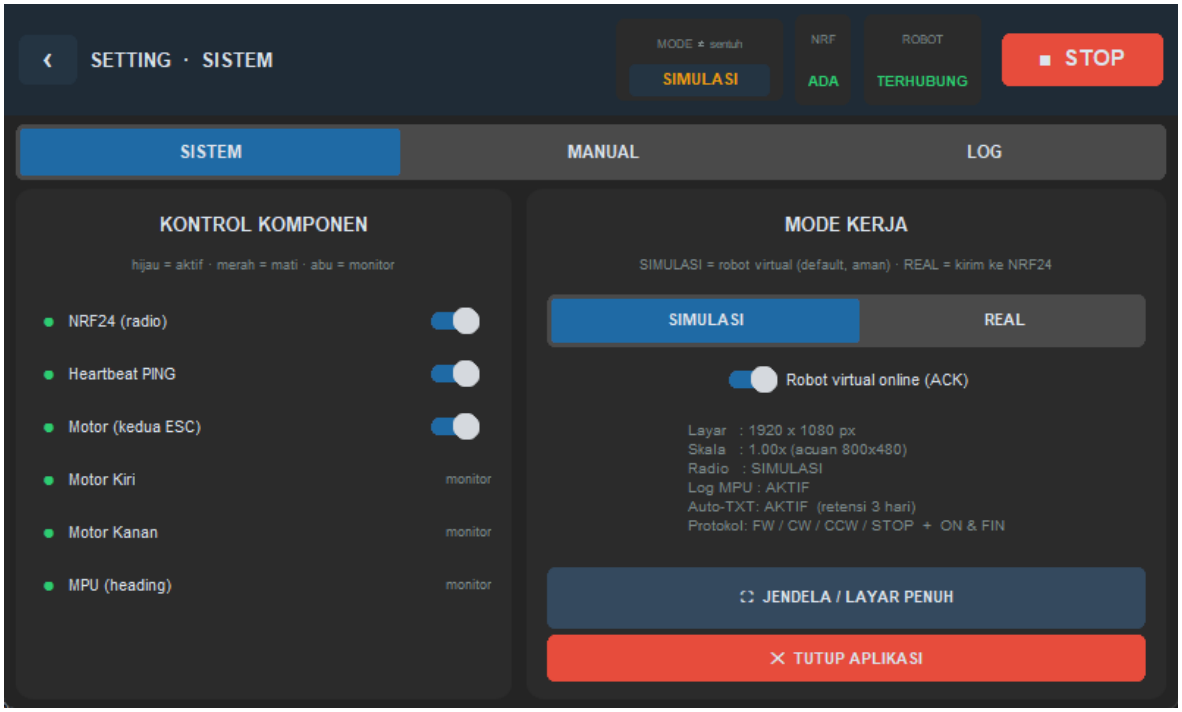
Gambar 4. SETTING › MANUAL: maju bersatuan meter, putar kiri/kanan dalam derajat, penunjuk rentang sah, dan tombol KIRIM per perintah.



Gambar 5. Papan angka layar penuh — operasi tanpa keyboard fisik, lengkap dengan penunjuk rentang nilai yang sah.



Gambar 6. SETTING › LOG: seluruh siklus protokol terekam beserta sudut MPU; auto-simpan TXT dan Log MPU sudah menyala sejak aplikasi dibuka.



Gambar 7. SETTING › SISTEM: saklar komponen, pemilih MODE KERJA, dan informasi resolusi layar beserta faktor skala tampilan.

6. Pengujian (Black-Box)

Uji	Skenario	Hasil Diharapkan
T-1	Input P=10 L=4, tekan KIRIM MISI	Antrean 29 perintah tersusun & dijalankan berurutan
T-2	Robot online (ACK) di simulasi	Status ROBOT = TERHUBUNG (hijau)

Uji	Skenario	Hasil Diharapkan
T-3	Robot offline	Status ROBOT = TERPUTUS (merah) < 3 s
T-4	Matikan saklar Motor	STOP:0 terkirim, antrean batal, LED motor merah
T-5	Input non-angka pada dimensi	Muncul dialog error, misi tidak dikirim
T-6	Tekan Export CSV/TXT	Berkas log tersimpan & berisi seluruh event
T-7	Seluruh perintah tereksekusi	Status = SELESAI setelah FIN perintah terakhir diterima
T-8	Nyalakan saklar Log MPU (tanpa mengatur port apa pun)	Roll/Pitch/Yaw tampil realtime dari telemetri radio; satu baris MPU per detik masuk log, tanpa baris kembar
T-9	Isi dimensi/lebar lintasan/titik awal	Kanvas & antrean perintah berubah realtime
T-10	Sentuh kolom input lalu pakai papan angka	Nilai kolom terisi tanpa keyboard fisik; preview ikut ter-update
T-11	Ganti titik awal (menu / sentuh kanvas)	Jalur & arah belok pertama berubah sesuai sudut awal
T-12	Kirim FW / CW / CCW dari halaman MANUAL	Satu paket terkirim; status ROBOT BERGERAK lalu SELESAI setelah FIN
T-13	Isi CW = 999 pada papan angka	Ditolak dengan pesan "nilai harus 1–180 derajat"
T-14	Tekan STOP saat antrean berjalan	STOP:0 terkirim, status DIBATALKAN, sisa antrean tidak dikirim
T-15	Robot tidak membalas ON dalam 1 detik	Status KIRIM ULANG; perintah dikirim ulang hingga 5 percobaan
T-16	Jalankan pada layar beresolusi berbeda	Jendela mengikuti ukuran layar; faktor skala tampil di halaman SISTEM
T-17	Buka aplikasi tanpa argumen apa pun	Chip MODE = SIMULASI; tidak ada paket terkirim ke radio nyata
T-18	Sentuh chip MODE di PC tanpa pyrf24	Muncul pesan sebabnya, mode kembali SIMULASI, chip & pemilih SISTEM tetap sinkron
T-19	Sentuh chip MODE saat antrean berjalan	Perpindahan ditolak dengan pesan "tekan STOP dulu"
T-20	Isi MAJU = 4,5 m lalu KIRIM	Terpecah menjadi 3 paket FW:150 berurutan; jumlah paket sudah tampil di kartu sebelum dikirim
T-21	Isi MAJU = 0,05 m	Ditolak: di bawah batas 0,1 m (paket FW terkecil 10 cm)
T-22	Nyalakan Auto-simpan TXT lalu jalankan misi	Berkas logs/log_feeder_*.txt terisi baris demi baris selagi misi berjalan
T-23	Tutup aplikasi paksa saat auto-simpan menyala	Berkas TXT tetap berisi seluruh baris hingga kejadian terakhir
T-24	Buka aplikasi, amati layar pembuka	Hanya dua tombol (MISI, SETTING); tidak ada bilah navigasi

Uji	Skenario	Hasil Diharapkan
T-25	Tekan KIRIM MISI	Aplikasi berpindah SENDIRI ke DASHBOARD; status & koordinat MPU berjalan realtime
T-26	Tekan HENTIKAN di DASHBOARD	STOP:0 terkirim, status DIBATALKAN, jalan pintas dashboard di BERANDA hilang
T-27	Taruh berkas log bertanggal 5 hari lalu di folder logs/, buka aplikasi	Berkas terhapus & tercatat di log; berkas sesi berjalan tetap ada

7. Panduan Instalasi & Penggunaan

7.1 Instalasi

```
# Di Raspberry Pi (kendali nyata):
pip install customtkinter pyrf24

# Di PC (mode simulasi, tanpa NRF):
pip install customtkinter
```

7.2 Menjalankan

Cukup jalankan satu berkas entry point **receiverArduino.py**; GUI, telemetri, dan logging berjalan otomatis:

```
cd Master Code/Transmitter
python receiverArduino.py # GUI layar penuh (layar sentuh 7")
python receiverArduino.py --windowed # GUI jendela 800x480 (uji di PC)
python receiverArduino.py --real # langsung memakai NRF sungguhan
python receiverArduino.py --cli # mode teks (opsional)
```

Mode awal selalu SIMULASI. Untuk mengendalikan robot sungguhan, sentuh chip **MODE** di bilah atas, pilih REAL pada **SISTEM** → **MODE KERJA**, atau jalankan dengan **--real**.

Pada mode layar penuh, tombol **Escape** atau **F11** mengembalikan tampilan ke mode jendela — berguna saat pengembangan di PC. Tombol yang sama tersedia pada halaman SISTEM.

7.3 Berkas Terkait

Berkas	Keterangan
receiverArduino.py	ENTRY POINT — jalankan ini -> GUI (opsi --windowed / --cli)
feeder_gui.py	Aplikasi GUI (CustomTkinter) — halaman bergaya ATM, mengikuti ukuran layar
feeder_core.py	Backend: RobotCommand, RadioLink (REAL/SIMULASI), CommandRunner, path planner, ActivityLogger (+auto-simpan), MPUMonitor
capture_gui.py	Skrip pengambil screenshot tiap halaman GUI untuk dokumentasi
build_docs_pdf.py	Generator dokumen PDF ini

Berkas	Keterangan
Robot/New/Finalnew/Finalnew.ino	Firmware robot (ESP32) — ditulis ulang sebagai eksekutor perintah FW/CW/CCW/STOP yang membalas ON & FIN
Robot/New/Finalnew/Dokumen Spesifikasi Teknis_Skripsi.docx	Acuan perilaku robot & protokol komunikasi
logs/	Berkas auto-simpan TXT — dibuat sendiri sejak aplikasi dibuka; isi > 3 hari dihapus otomatis

7.4 Deploy di Raspberry Pi (rekomendasi)

GUI dijalankan di Raspberry Pi karena memiliki GPIO/SPI untuk modul NRF24. GUI membutuhkan sesi **Desktop** (monitor HDMI atau VNC) — tidak bisa lewat SSH polos. Wiring NRF24L01 mengikuti pemetaan pin berikut (kode: RF24(24, 0) -> CE=GPIO24, CSN=CE0):

NRF24L01	Raspberry Pi (BCM)	Pin fisik
VCC (3.3V)	3.3V	1 / 17
GND	GND	6, 9, ...
CE	GPIO24	18
CSN	GPIO8 (CE0)	24
SCK	GPIO11	23
MOSI	GPIO10	19
MISO	GPIO9	21

Catatan: VCC NRF wajib 3.3V (bukan 5V); tambahkan kapasitor 10µF antara VCC–GND modul agar catu daya stabil.

Instalasi otomatis — ekstrak paket **raspberry_feeder.zip**, lalu jalankan skrip setup sekali:

```
cd ~/feeder # lokasi hasil ekstrak
chmod +x setup_pi.sh run.sh
./setup_pi.sh # aktifkan SPI, apt, venv, pip install
```

Instalasi manual (bila ingin bertahap):

```
sudo raspi-config nonint do_spi 0 # aktifkan SPI
sudo apt install -y python3-venv python3-tk
sudo usermod -aG spi,gpio,dialout $USER # lalu logout-login
python3 -m venv --system-site-packages ~/feeder-venv
source ~/feeder-venv/bin/activate
pip install -r requirements.txt
```

Menjalankan:

```
./run.sh # buka GUI
./run.sh --cli # mode teks lama
```

Catatan: Gunakan venv dengan --system-site-packages agar python3-tk (dari apt) terlihat di dalam venv; tanpa itu GUI gagal import tkinter. Bila status chip MODE menolak berpindah ke REAL di Pi, berarti SPI belum aktif / wiring NRF salah (cek: ls /dev/spidev*) — sebabnya ditampilkan pada dialog dan log.

8. Kesimpulan & Iterasi Berikutnya

Prototipe GUI berhasil memenuhi seluruh kebutuhan fungsional sesuai lingkup iterasi ini, dengan pemisahan arsitektur yang bersih dan mode simulasi yang mempermudah evaluasi. **Iterasi ini** — menindaklanjuti umpan balik pembimbing — menghasilkan perubahan pokok berikut:

- **Tampilan mengikuti ukuran layar perangkat** dan dirancang ulang bergaya mesin ATM untuk layar sentuh 7 inci: enam halaman bertombol besar, tombol STOP permanen, dan papan angka layar penuh — **tanpa menghilangkan satu pun fitur**.
- **Perilaku robot mengikuti Dokumen Spesifikasi Teknis**: protokol Request-Response "[Moves]:[Dist]" dengan sinyal balik ON dan FIN, lengkap dengan pengiriman ulang otomatis bila ON tidak diterima dalam 1 detik.
- **Pola jalur berpindah ke Pengendali Utama.** Robot kini menjadi eksekutor satu perintah; kemajuan misi yang ditampilkan adalah konfirmasi nyata dari robot, bukan perkiraan berbasis penghitung waktu.
- **Kendali manual FW / CW / CCW** melengkapi STOP yang sudah ada, dengan validasi rentang sesuai tabel perintah.
- **Satuan antarmuka mengikuti cara operator berpikir:** perintah maju manual diisi dalam **meter** dan dipecah otomatis menjadi paket FW 10–200 cm, sehingga protokol tetap utuh tanpa membebani operator menghitung sentimeter.
- **MPU lepas dari port serial:** sudutnya ikut telemetri radio dan mengalir di latar belakang, dengan saklar **Log MPU** untuk mencatatnya — satu bidang input beserta satu dependensi (pyserial) berhasil dihapus.
- **Mode kerja menjadi keputusan operator:** REAL/SIMULASI dapat dipindah kapan saja lewat chip MODE, dengan SIMULASI sebagai keadaan awal yang aman.
- **Auto-simpan log ke TXT** menyala sejak aplikasi dibuka sehingga catatan uji lapangan tetap utuh tanpa perlu mengingat menekan tombol Export, sementara berkas berumur lebih dari 3 hari dibuang sendiri agar kartu SD Raspberry Pi tidak penuh.
- **Navigasi berubah dari menu menjadi alur:** beranda menyisakan MISI dan SETTING, dan DASHBOARD — berisi status, koordinat MPU, serta tombol berhenti — terbuka sendiri begitu misi dikirim.

Sebagai efek samping yang menguntungkan, belokan kini **ditutup memakai umpan balik sudut yaw MPU** (bukan durasi tetap), sehingga rencana iterasi sebelumnya soal penghapusan kalibrasi TURN_90_MS sudah terpenuhi. Sesuai Prototyping Model, keterbatasan yang tersisa menjadi rencana iterasi selanjutnya:

- Memasang **sensor RPM (hall/encoder)** untuk mengganti estimasi dengan data ukur.
- Menambah **odometri** agar jarak FW tidak lagi diturunkan dari waktu tempuh, sehingga tahan terhadap arus air.
- Memperluas **isi sinyal balik** (status komponen, throttle asli, tegangan baterai) di atas jalur 2 arah yang sudah ada.
- Menambah perintah untuk **kontrol komponen individual** (motor kiri/kanan, MPU).
- Mendukung **path waypoint bebas** (sentuh banyak titik) selain sapuan persegi.
- Kalibrasi konstanta (ROBOT_SPEED_MPS, TURN_RATE_DPS, KV) pada pengujian air.
